COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	SECTION:	CPE21S2
DATE PERFORMED:	09-18-26	DATE SUBMITTED:	09-23-26
NAME:	Christian Lee C. Bandigan	INSTRUCTOR:	Engr. Jimlord M. Quejado

ACTIVITY NO. 9.1 Hands-on Activity 9.1 Tree ADT

1. PROCEDURE OUTPUT

Code + Console Screenshot

```
template <typename T>
class GeneralTree{
    private:
        T key;
        std::vector<GeneralTree<T>*> children;

    public:
        GeneralTree(T key);
        void insertChild(T newNode);

        GeneralTree<T>* getChild(T nodee);
};

template  <typename T>
GeneralTree<T>::GeneralTree(T value){
    key = value;
}

template <typename T>
void GeneralTree<T>::insertChild(T newNode){
    children.push_back(new GeneralTree<T>(newNode));
}

template<typename T>
GeneralTree<T>* GeneralTree<T>::getChild(T node){
    for(int i =0; i<children.size();i++){
        if(children[i]->key == node){
            return children[i];
        }
    }
    return nullptr;
}
```



```
int main(){
    GeneralTree<char>* root = new GeneralTree<char>('A');

    root->insertChild('B');
    root->insertChild('C');
    root->insertChild('D');
    root->insertChild('E');
    root->insertChild('F');
    root->insertChild('G');

    root->getChild('D')->insertChild('H');
    root->getChild('E')->insertChild('I');
    root->getChild('E')->insertChild('J');
    root->getChild('F')->insertChild('K');
    root->getChild('F')->insertChild('L');
    root->getChild('F')->insertChild('M');
    root->getChild('G')->insertChild('N');
    root->getChild('E')->getChild('J')->insertChild('P');
    root->getChild('E')->getChild('J')->insertChild('Q');

```

Observations	Basically the same code for the binary tree but I used a vector to have multiple children.

Table 9-1. General Tree Implementation

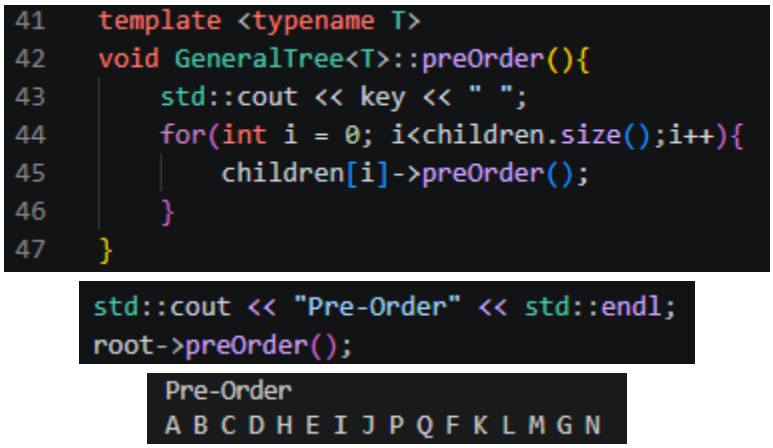
Node	Height	Depth
A	3	0
B	0	1
C	0	1
D	1	1
E	2	1
F	1	1
G	1	1
H	0	2
I	0	2
J	1	2

K	0	2
L	0	2
M	0	2
N	0	2
P	0	3
Q	0	3

Table 9-2. Height and Depth of Nodes

Pre-Order	A, B, C, D, H, E, I, J, P, Q, F, K, L, M, G, N
Post-Order	B, C, H, D, I, P, Q, J, E, K, L, M, F, N, G, A
In-Order	B, A, C, H, D, I, E, P, J, Q, K, F, L, M, N, G

Table 9-3. Manual Traversal

Code + Console Screenshot	
Observations	To print the tree in pre-order the function prints the key first, uses a for loop to traverse the children vector and recursively calls the function on the children. The output from this function is the same when I manually traversed it in table 9-3.

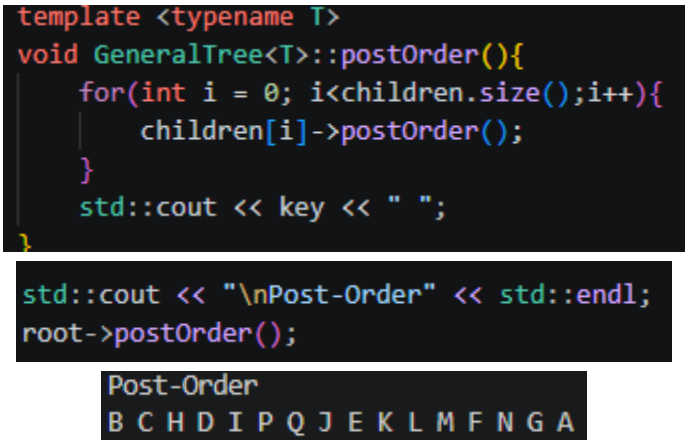
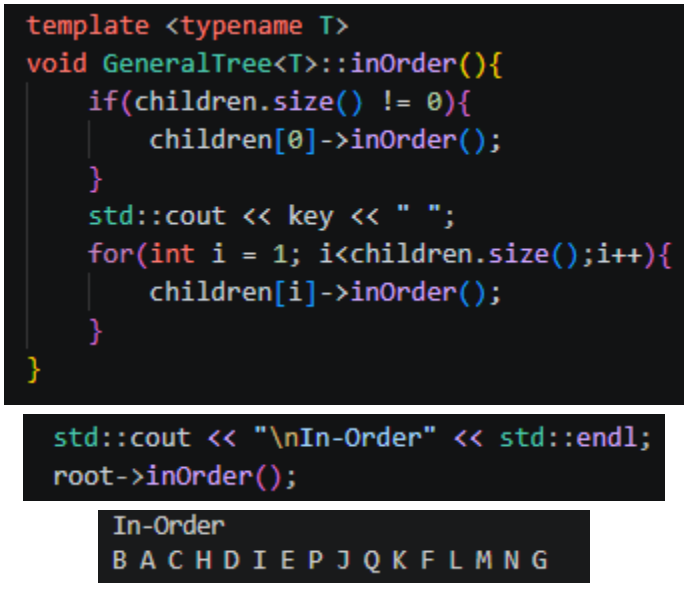
Code + Console Screenshot	
Observations	To print the tree in post-order function uses a for loop to traverse the children vector and recursively calls the function on the children before printing. The output from this function is the same when I manually traversed it in table 9-3.
Code + Console Screenshot	
Observations	To print the tree in in-order function uses a for loop to traverse the children vector and recursively calls the function on the children before printing. The output from this function is the same when I manually traversed it in table 9-3.

Table 9-4. Traversal Functions

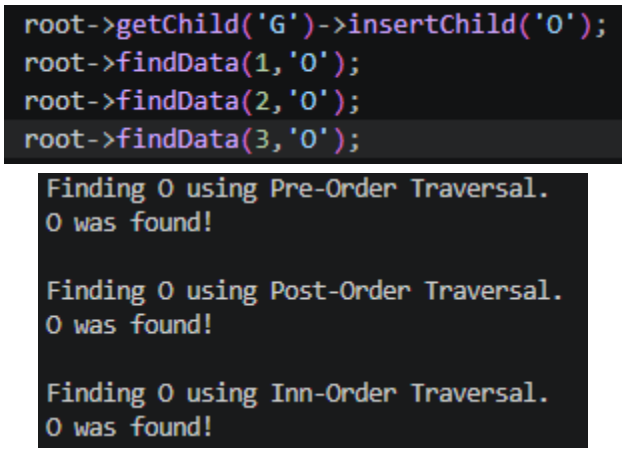
Code

```
73  bool GeneralTree<T>::preOrder(T node){
74      if(key == node){
75          return true;
76      }
77      for(int i = 0; i<children.size();i++){
78          if(children[i]->preOrder(node)){
79              return true;
80          }
81      }
82
83      return false;
84  }
85  template <typename T>
86  bool GeneralTree<T>::postOrder(T node){
87      for(int i = 0; i<children.size();i++){
88          if(children[i]->postOrder(node)){
89              return true;
90          }
91      }
92      if(key == node){
93          return true;
94      }
95
96      return false;
97  }
98  template <typename T>
99  bool GeneralTree<T>::inOrder(T node){
100     if(children.size() != 0){
101        if(children[0]->inOrder(node)){
102            return true;
103        }
104    }
105    if(key == node){
106        return true;
107    }
108    for(int i = 1; i<children.size();i++){
109        if(children[i]->inOrder(node)){
110            return true;
111        }
112    }
113
114    return false;
115 }
```

	
Analysis	To create a find data function, I created and modified my traversal functions so that it returns a bool function once the key is found and these functions are then called in the findData function based on what traversal the user wants..

```
119  template <typename T>
120  void GeneralTree<T>::findData(int choice, T node){
121      bool found = false;
122      if(choice == 1){
123          std::cout <<std::endl;
124          std::cout << "Finding " << node <<" using Pre-Order Traversal.\n";
125          found = preOrder(node);
126      }else if(choice == 2){
127          std::cout <<std::endl;
128          std::cout << "Finding " << node <<" using Post-Order Traversal.\n";
129          found = postOrder(node);
130      }else if(choice == 3){
131          std::cout <<std::endl;
132          std::cout << "Finding " << node <<" using Inn-Order Traversal.\n";
133          found = inOrder(node);
134      }else{
135          std::cout <<std::endl;
136          std::cout << "Invalid input\n";
137      }
138
139      if(found){
140          std::cout<< node << " was found!"<< std::endl;
141      }else{
142          std::cout<< node << " was not found"<< std::endl;
143      }
144
145  }
```

Table 9-5. findData function

Code + Console Screenshot	
Observations	It successfully inserted the O node as the child for node G and found that node using the function and the three traversals.

```
root->getChild('G')->insertChild('O');
root->findData(1,'O');
root->findData(2,'O');
root->findData(3,'O');
```

```
Finding O using Pre-Order Traversal.
O was found!

Finding O using Post-Order Traversal.
O was found!

Finding O using Inn-Order Traversal.
O was found!
```

Table 9-6. Test findData function

.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Step 1:

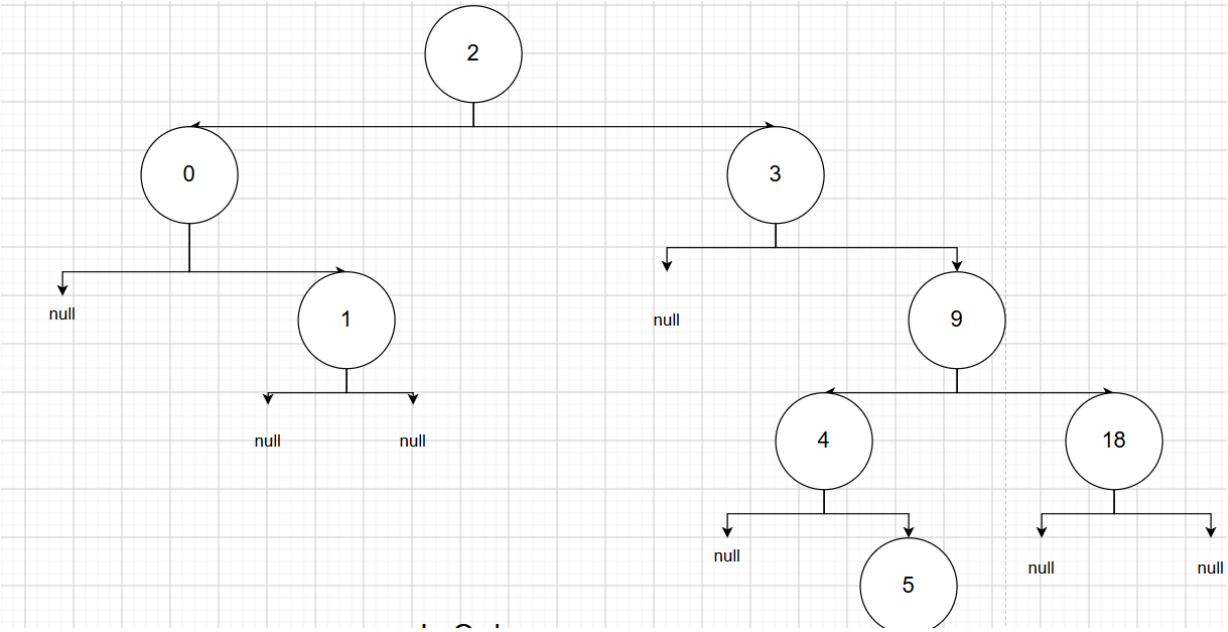
```
5  class BinarySearchTree{
6      private:
7          //create a key
8          T key;
9
10         //pointers for left and right
11         BinarySearchTree<T>* left;
12         BinarySearchTree<T>* right;
13     public:
14         //constructor
15         BinarySearchTree(T value);
16
17         //insert left and right
18         void insert(T value);
19
20         //traversing
21         void preOrder();
22         void inOrder();
23         void postOrder();
24         //searching
25
26 };
27 template <typename T>
28 BinarySearchTree<T>::BinarySearchTree(T value){
29     key = value;
30     left = nullptr;
31     right = nullptr;
32 }
33 template <typename T>
34 void BinarySearchTree<T>::insert(T value){
35     //left or right
36
37     //go left if value is < key insert to left
38     if(value < key){
39         if(left == nullptr){
40             left = new BinarySearchTree<T>(value);
41         }else{
42             left->insert(value);
43         }
44     //go right if value > key insert to right
45     }else if(value > key){
46         if(right == nullptr){
47             right = new BinarySearchTree<T>(value);
48         }else{
49             right->insert(value);
50         }
51     }
```

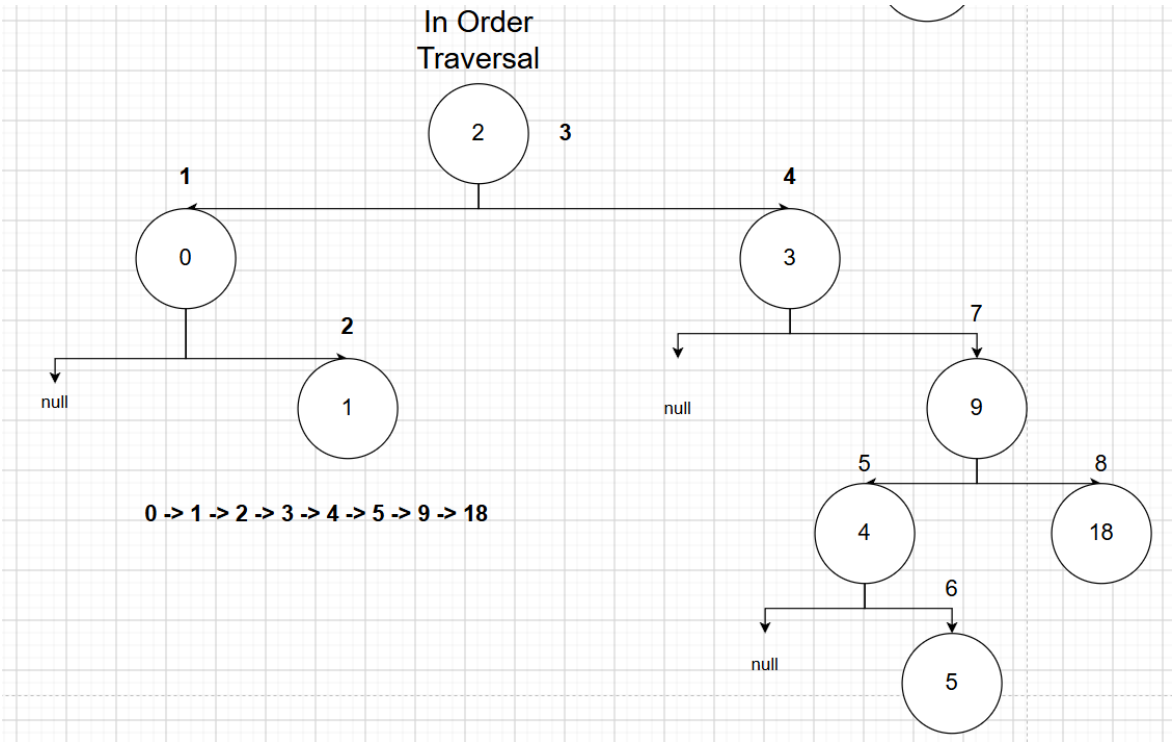
```cpp
#include <iostream>
#include "BST.h"

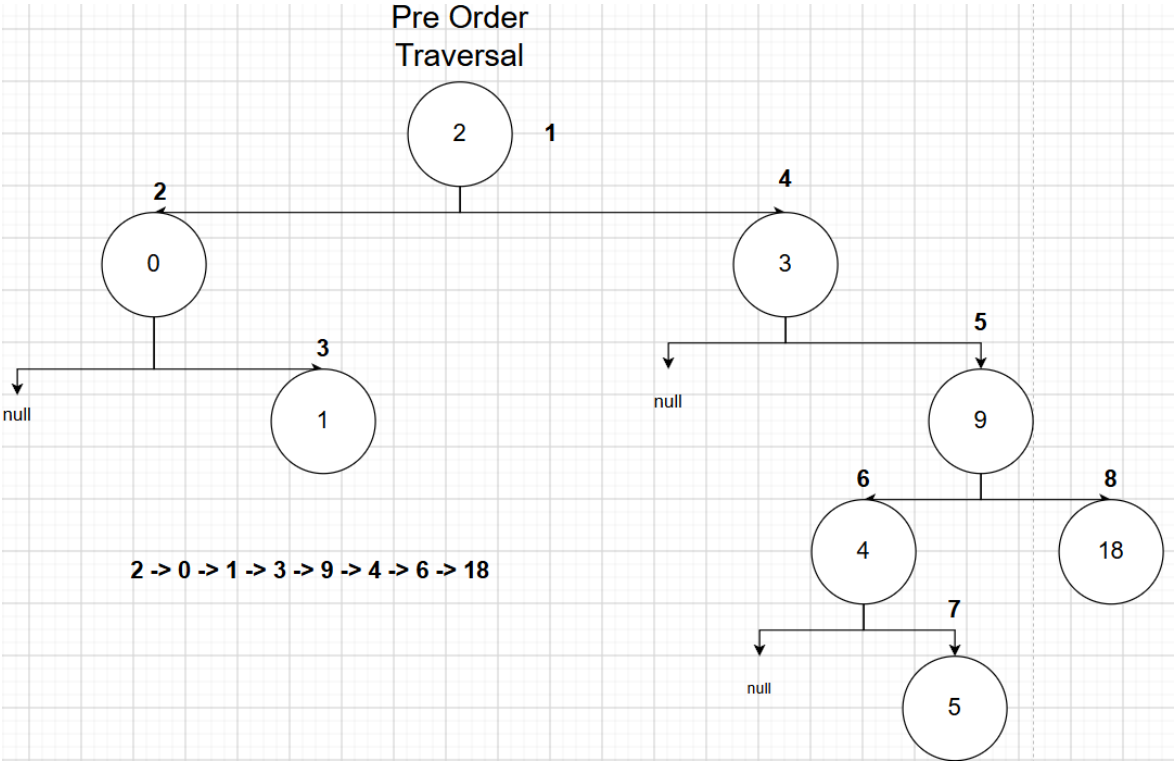
int main(){
    //create the root node
    BinarySearchTree<int> tree(2);

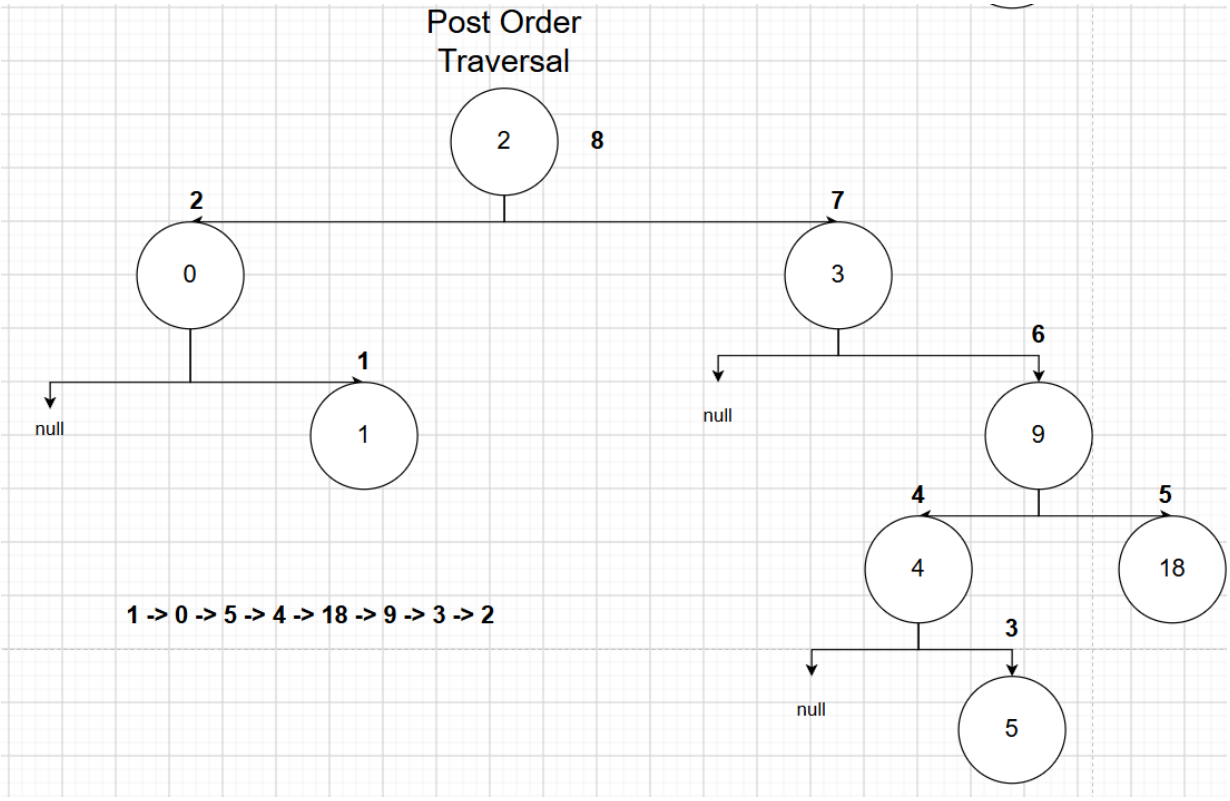
    tree.insert(3);
    tree.insert(9);
    tree.insert(18);
    tree.insert(0);
    tree.insert(1);
    tree.insert(4);
    tree.insert(5);
```

Step 2: Tree Diagrams




In Order Traversal
2
3
1
4
0
3
2
7
null
1
null
9
5
8
4
18
6
null
5
0 -> 1 -> 2 -> 3 -> 4 -> 5 -> 9 -> 18


Pre Order Traversal
2
1
2
4
0
3
3
5
null
1
null
9
6
8
4
18
7
null
5
2 -> 0 -> 1 -> 3 -> 9 -> 4 -> 6 -> 18



Step 3:

In-Order Traversal

```
67  template <typename T>
68  void BinarySearchTree<T>::inOrder(){
69      if(left != nullptr){
70          left->inOrder();
71      }
72      std::cout << key << " ";
73      if(right != nullptr){
74          right->inOrder();
75      }
76  }
```

```
inOrder: 65 66 69 72 75 77 80 83 84
```

The function worked in printing the tree in order. The output is the same with my sequence in the diagram.

Pre-Order Traversal

```
55  template <typename T>
56  void BinarySearchTree<T>::preOrder(){
57      std::cout << key << " ";
58      if(left != nullptr){
59          left->preOrder();
60      }
61
62      if(right != nullptr){
63          right->preOrder();
64      }
65  }
```

```
preOrder: 75 69 66 65 72 80 77 84 83
```

The function worked in printing the tree in pre-order. The output is the same with my sequence in the diagram.

Post-Order Traversal

```
78  template <typename T>
79  void BinarySearchTree<T>::postOrder(){
80      if(left != nullptr){
81          left->postOrder();
82      }
83      if(right != nullptr){
84          right->postOrder();
85      }
86      std::cout << key << " ";
87  }
```

```
postOrder: 65 66 72 69 77 83 84 80 75
```

The function worked in printing the tree in post-order. The output is the same with my sequence in the diagram.

Conclusion

In this activity I learned about trees which is a non linear data structure that starts from a root node. I created two C++ implementations, general tree and binary tree. General trees can have multiple children while binary trees can have at most two children. I was also able to create functions that can insert nodes and traverse the whole tree in three ways. Moreover, in the supplementary activity, I was able to create a binary search tree which is a type of binary tree that inserts nodes whether it is greater or less than the current nodes in the tree.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

COMPUTER ENGINEERING

QUEZON CITY

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.